2026/27/1 XK85DZ-IP-18fTKHG - Telecommunication networks: Cinema Socket Sample Exam

Copying anyone else's solution is not allowed, nor is discussing solutions through a separate channel, using chat applications, or using any AI-based system, etc. You must solve the tasks on your own! The tasks and solutions may not be published in any form (email, Facebook, GitHub, forums, etc.)!

In case of obvious copying, everyone involved will receive a failing grade on the exam, regardless of who submitted the solution first! Apart from this, all other resources may be used (internet, documentation, lecture slides, notes, etc.).

Every step and message should also be printed to the console, so that the program's operation is visible!

For each subtask, attach to your submission the sample files you created for running it, and take a screenshot showing how the program was started (e.g. command-line arguments) and the final result, demonstrating that the program works (fully or partially)!

Cinema Sample

Create a cinema ticket purchasing server-client application!

Subtask 1 (4 points)
Create a server-client application that uses the TCP protocol.
The client starts the communication by sending the bytestring GET_SCREENINGS to the server. In response, the server sends the list of that day's screenings as a plain bytestring (there are at least 3 screenings each day). A screening consists of a film title, a hall name and a start time (as a string in "hh:mm" format). Screenings are separated by | characters, and the fields of a screening by semicolons.

Example response:
Dune;Hall 1;17:30|Oppenheimer;Hall 2;19:00|Barbie;Hall 1;20:45

Next, the client attempts to buy a ticket: it sends the server a film title and a start time as a bytestring, separated by a semicolon.

Example request:
Dune;17:30

Since this is a small cinema, each screening has only a few seats (e.g. five), so the server's response depends on seat availability: "SUCCESS" if there are still seats left for the screening, "FAIL" if there are none. The server sends its response using struct (7s6s), which contains the result and a ticket ID. The ticket ID is 6 random characters on a successful purchase and six zeros on failure.

Example responses:
SUCCESS, K7Q2XA (successful purchase)
FAIL, 000000 (failed purchase)

The client prints whether the purchase was successful, along with the received ticket ID.

Example client output:
Sending: GET_SCREENINGS
Received screenings:
 Dune, Hall 1, 17:30
 Oppenheimer, Hall 2, 19:00
 Barbie, Hall 1, 20:45
Buying ticket: Dune 17:30
Purchase successful, ticket ID: K7Q2XA

Subtask 2 (1 point)
The server reads the list of screenings from a file whose path is given as a command-line argument. For each screening, the file also contains the number of free seats. The server reads these and keeps track of them, decreasing the number of free seats after each successful purchase.

Example:
python cinema_server.py screenings.txt

Example screenings.txt:
Dune;Hall 1;17:30;5
Oppenheimer;Hall 2;19:00;3
Barbie;Hall 1;20:45;0

Subtask 3 (1 point)
The server must be able to serve multiple clients (using select).

Subtask 4 (4 points)
A ticket scalper disrupts ticket purchasing with a man-in-the-middle attack.
Write a proxy that sits between the server and the client. The proxy only needs to handle one client at a time (clients connect to it one after another). The list of screenings (GET_SCREENINGS) and the purchase request are simply relayed between the client and the server unchanged. To process the server's response to the purchase, the proxy uses the same struct as the client and the server (7s6s). If the server reports a failed purchase (FAIL, 000000), the proxy simply forwards it to the client and prints this as well. For every successful purchase, with 50% probability it forwards the ticket ID, and with 50% probability it keeps the ticket for itself and reports a failed purchase to the client (FAIL, 000000).
The proxy continuously prints the IDs of the successfully stolen tickets.

Example proxy output:
Forwarded ticket: K7Q2XA
Stolen ticket: P3MZ8D
Forwarded failed purchase: FAIL, 000000

Stolen ticket: 4TB9WE